

MIDTERM EXAM



Name: [REDACTED]

Question One

(5 Points)

Show the results of the following sequence of events (see table below), by drawing the state of the data structure where add and remove are the operations that correspond to the basic operations of each data structure.

		Stack	Queue
1	Add(4)		
2	Add(8)		
3	Add(6)		
4	Remove()		
5	Remove()		

Question Two

(8 Points)

For questions 1 and 2, consider a linear queue of integer elements declared as follows:

```
#define Max 20
typedef int queue_type[Max]
```

Where:

- The first element in the queue is usually kept in the first cell of the array, which means, when an element is dequeued all other elements shifted up.
- The index of the last element in the queue is kept in the last cell of the array.
- The queue is initialized by assigning -1 to the last cell in the array

1. Write a C function enqueue to insert an element to an initialized and not full queue.

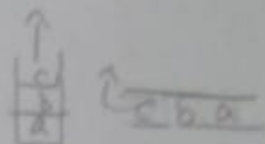
```
void enqueue(EntryType item, queue_type *q) {
    if ((*q)[Max-1] == -1) { (*q)[0] = item; (*q)[Max-1] = 0; }
    else { int i;
        for (i = 0; i < (*q)[Max-1]; i += 2) { int temp;
            temp = (*q)[i+1];
            (*q)[i+1] = (*q)[i];
            (*q)[i+2] = temp;
        } //end for
        (*q)[0] = item;
    } //end else
} //end Function
```

2. Write a C function `is_queue_full` to check if the queue is full or not.

```
int is_queue_full(queue_type q){
```

```
    return q[Max-1] == Max-2 ;
```

```
}
```



Question Three

(7 Points)

abc: cba

Without using an array, write a program that will read one line of input from the terminal. The input is supposed to consist of two parts separated by colon ":". Your program should produce a single character as follows:

N No colon on the line

D The left and right parts have the same length but are different

M The left is a mirror of the right part

22

```
void main() { char c, d; StackType S; QueueType q;
CreateStack(&S); CreateStack(&S); CreateQueue(&q);
while (getchar() != '\n') { Push(c, &S); Push(c, &S1);
    Enqueue(c, &q); }
while (pop(&c, &S1)) { if (c == ':') {
    printf("N\n"); return 0; }
int flag1 = 0;
while (pop(&c, &S)) { dequeue(&c1, &q);
    if (c != c1) { flag1 = 1; break; }
} if (IsEmptyQueue(q) && IsEmptyStack(S))
{ if (flag1 == 0) {
    printf("M\n"); return 0; }
else { printf("D\n"); return 0; }
}
```